F O R M A L C H O P T U I K

Анализ пробелов и выбор системы доказательств

Аудит программы формализации Чоптуика: реестр G1–G6 и сравнение Isabelle/HOL · Coq/Rocq · Lean 4 · том II

Автор: Исаев Исак Хамзатович · ORCID 0009-0003-7299-0701

Программа: choptuik_ac_bc · крыло формальной верификации

Нальчик · 2026

Аннотация

Монография представляет полный аудит программы формализации критических явлений Чоптуика. Фиксируется классифицированный реестр из шести пробелов G1–G6 с уровнями критичности от Blocker до Low: существование критического решения, спектральная теорема устойчивости, геометрия изотропной сингулярности, типизированный мост Lean–Python, интервальная арифметика и аксиоматическая база. Центральный блокер — существование — разбирается до уровня стратегии: ньютоновская коррекция численного профиля в банаховом пространстве последовательностей с экспоненциальным затуханием, компактный обратный оператор и количественная теорема о неявной функции.

Проводится полное сравнительное исследование систем доказательств Isabelle/HOL, Coq/Rocq и Lean 4 по критериям логики, библиотечного покрытия, автоматизации и интеграции с Python; обосновывается выбор Lean 4 с mathlib. Формулируется техническое задание: среда Lean 4 + Lake, модульная организация Choptuik.*, дисциплина CI с аудитом аксиом и sorried-фрагментов, аксиоматическая база из трёх стандартных аксиом (Classical.choice, propext, Quot.sound) и регрессионная блокировка протокола моста.

Примечание о состоянии программы. После написания этого тома программа продвинулась: дерево теории свободно от обязательств (ни одного sorry) и чисто по аксиомам, интервальная арифметика (гэп G5) закрыта с доказанными теоремами корректности, аксиоматическая база (G6) зафиксирована и проверяется в CI, а мост Lean⇄Python несёт живой канал сертификатов, чьи точные рациональные артефакты проверяются в Lean и разряжаются в итоговые теоремы. Настоящий том сохраняет исходную перспективу дорожной карты; ревизионный снимок текущего состояния дан в Томе III серии «Канал сертификатов и архитектура доверия».

Ключевые слова: анализ пробелов; Lean 4; Isabelle/HOL; Coq/Rocq; mathlib; теорема о неявной функции; компактные операторы; интервальная арифметика; аксиоматика; формальная верификация.

Содержание

1. Введение: цель и метод аудита....1
1.1. Критичность как функция внешних обязательств....1
1.2. Связь с томом I и дорожной картой....1
2. Текущее состояние программы choptuik_ac_bc....1
2.1. Чего не хватает для формализации....2
2.2. Дисциплина честных оговорок....3
3. Классифицированный реестр пробелов....3
3.1. Физико-математические пробелы: G1, G2, G3....3
3.2. Инструментальные пробелы: G4, G5....4
3.3. Управленческий пробел: G6....4
4. Центральный блокер: существование критического решения....5
4.1. Стратегия Рейтерера–Трубовица....5
4.2. Что требуется в Lean....5
4.3. DSS-периодичность как часть утверждения....6
5. Устойчивость: требования спектральной теории....6
5.1. Аттрактор кодименсии один....7
5.2. Необходимый аппарат и его статус в mathlib....7
5.3. От спектра к наблюдаемым: γ, универсальность, эхо-формула....8
6. Сравнительный анализ: Isabelle/HOL, Coq/Rocq, Lean 4....8
6.1. Isabelle/HOL: скорость автоматизации, простые типы....9
6.2. Coq/Rocq: алгебраическая сила, ценой кривой входа....9
6.3. Lean 4: mathlib, нативная автоматизация и мост к Python....9
6.4. Условия пересмотра решения....11
7. Техническое задание и аксиоматическая база....11
7.1. Организация модулей....12
7.2. Аксиоматическая база....12
7.3. Интеграция и контроль качества....12
8. Выводы и рекомендации....13
Список литературы....14

Примечание: оглавление построено на кодах полей. После редактирования документа обновите номера страниц: щёлкните по оглавлению правой кнопкой мыши и выберите «Обновить поле».

1. Введение: цель и метод аудита

Настоящий том — аудиторский документ программы формализации. Его задача двоякая: во-первых, дать классифицированную опись всего, что отделяет текущее состояние вычислительной программы choptuik_ac_bc от машинно-проверяемой теории критических явлений Чоптуика; во-вторых, зафиксировать и обосновать решение о выборе системы доказательств. Если том I отвечает на вопрос «как строить», то этот том — на вопросы «что именно отсутствует» и «каким инструментом строить».

Метод аудита прост и воспроизводим. Каждый потенциальный пробел формулируется как утверждение вида «в Lean не существует доказательства P» или «не существует типизированного канала Q»; затем пробел классифицируется по критичности (Blocker, High, Medium, Low), по зависимостям от других пробелов и по этапу дорожной карты, на котором его закрытие запланировано. Итог — реестр из шести позиций G1–G6, машиночитаемая версия которого хранится в blueprint/gap_registry.csv и синхронизирована с тегами sorried-фрагментов в исходном коде Lean.

1.1. Критичность как функция внешних обязательств

Классификация по критичности привязана не к субъективной сложности, а к внешним обязательствам программы. Пробел является Blocker'ом, если без его закрытия ни одно из трёх публичных утверждений программы — существование критического решения, единственность неустойчивой моды, закон масштабирования с $\gamma = 1/\lambda_0$ — не может быть заявлено как машинно-проверяемое. Пробел уровня High необходим для полноты теории, но допускает частичные промежуточные результаты. Уровни Medium и Low — это инфраструктура и управление: их незакрытость замедляет работу, но не обесценивает её результаты.

1.2. Связь с томом I и дорожной картой

Реестр пробелов и дорожная карта — два среза одной и той же программы: реестр упорядочивает по критичности, карта — по времени. Каждой позиции реестра соответствует стратегия закрытия, фаза и задачи дорожной карты; каждая задача карты ссылается на пробел, который она приближает. Документы синхронизированы: изменение реестра без обновления карты и наоборот не проходит ревью. Такая двойная бухгалтерия — цена за честность: у программы нет «тихих» обязательств.

2. Текущее состояние программы choptuik_ac_bc

Честный аудит начинается с описания того, что уже есть. Родительский репозиторий — не набор скриптов, а сложившаяся исследовательская программа с восемью направлениями, из которых для формализации критических явлений релевантны четыре: прямая работа с уравнениями Эйнштейна (Einstein Direct), вычислительная лаборатория динамики инвариантов (dsi_lab), интерактивная визуализация и блок верификации с тестовой инфраструктурой. Ключевой актив — дисциплина воспроизводимости: данные хранятся в JSON, вычисления детерминированы, результаты сопровождаются тестами и отчётами о верификации.

Второй актив — опыт обнаружения ошибок в опубликованной литературе. Программа находила расхождения в формулах классических работ по критическому коллапсу; это означает, что численный контур откалиброван достаточно тонко, чтобы служить арбитром, — редкое свойство для вычислительной физики. Третий актив — единая формула Чоптуика и поправки к ней: аналитические конструкции, которые формализация должна будет либо вывести, либо опровергнуть, — готовая цель для машинной проверки.

2.1. Чего не хватает для формализации

При этом ни один фрагмент программы сегодня не является формализованной математикой. Отсутствуют строгие определения начальных данных, фазового пространства и гиперповерхности порога; нет формализации понятия критического решения и его самоподобия; нет спектральной теории для оператора возмущений; нет связи с любой системой доказательств. Python-код воспроизводит числа, но не несёт сертификатов корректности, приемлемых для доказательного ядра. Эти дефициты и составляют реестр главы 3.

Таблица 1. Активы и дефициты программы

Актив	Статус	Как используется формализацией
Воспроизводимые численные данные	есть	источник калибровки и кросс-проверок теорем
Опыт поиска ошибок в литературе	есть	прецедент строгости; мотивация формализации
Единая формула Чоптуика и поправки	есть	цель для вывода или опровержения в Lean
Строгие определения порога и критического решения	нет	создаются в этапе I–II (G1)
Спектральная теория оператора возмущений	нет	создаётся в этапе III (G2)
Канал Lean ↔ Python	нет	создаётся в этапе I (G4, G5)

2.2. Дисциплина честных оговорок

Существенно, что аудит не понижает статус программы, а уточняет его. Формулировка «гипотеза → машинное подтверждение → честные оговорки», принятая в родительском репозитории, означает, что каждое численное утверждение сопровождается явной формулировкой границ применимости. Формализация наследует эту дисциплину в более сильной форме: граница применимости превращается в набор аксиом и гипотез теоремы, которые ядро системы проверяет при каждом использовании. Там, где численная программа говорит «в этих пределах мы это видели», формализованная теория говорит «при этих гипотезах это доказано».

3. Классифицированный реестр пробелов

Реестр состоит из шести позиций. Две — блокеры, две — высокого уровня, одна — среднего, одна — низкого. В этой главе каждая позиция разбирается по единой схеме: формулировка, почему это пробел, от чего зависит, стратегия закрытия. Итоговая матрица критичности приведена на рисунке 1.

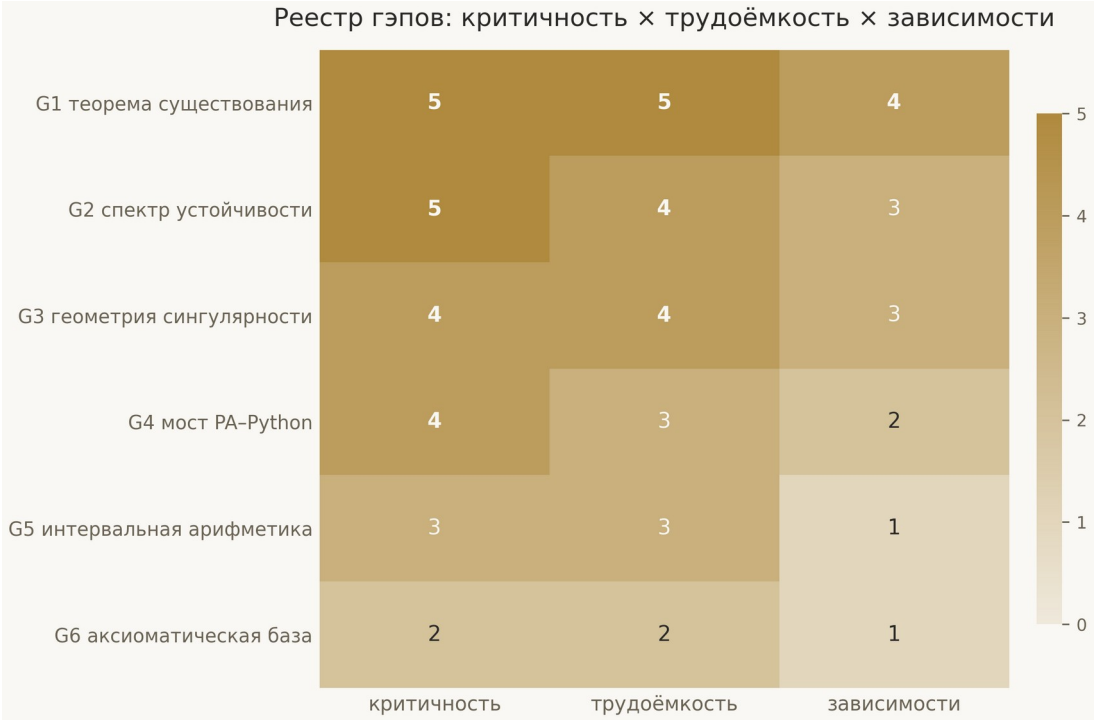


Рис. 1. Матрица реестра: критичность, трудоёмкость и глубина зависимостей для G1–G6 (шкала 0–5).

3.1. Физико-математические пробелы: G1, G2, G3

G1 (Blocker) — отсутствие строгого доказательства существования решения Чоптуика. Компьютерно-ассистированное доказательство Рейтерера–Трубовица существует, но не является машинно-проверяемым; его перенос в Lean — центральная задача этапа II. G2 (Blocker) — отсутствие формализации теории устойчивости: спектрального анализа линеаризованного оператора, доказательства единственности неустойчивой моды и вывода $\gamma = 1/\lambda_0$. Это ядро динамической картины критических явлений. G3 (High) — геометрия сингулярности: сильность через расходимость интеграла кривизны вдоль нулевой геодезической, изотропность через ограниченность Вейля, конформная компактификация. mathlib даёт многообразия и связности, но не даёт готовых нулевых геодезических критериев — слой нужно достраивать.

3.2. Инструментальные пробелы: G4, G5

G4 (High) — мост между формальным доказательством и численными расчётами. Без типизированного канала численные данные не могут входить в доказательства, а формальные константы не могут проверяться численно. Решение — ZeroMQ-протокол с семью методами и регрессионной блокировкой изменений; инженерная часть закрывается на этапе I. G5 (Medium) — интервальная арифметика: строгие оценки погрешностей, наследующие идеологию целочисленных сертификатов Рейтерера–Трубовица. В Python-крыле службы уже реализован класс Interval с направленным округлением через math.nextafter; в Lean требуется доказать корректность операций — задача этапа I, закрывающаяся вместе с mul_sound.

3.3. Управленческий пробел: G6

G6 (Low) — фиксация аксиоматической базы. Система Lean по умолчанию допускает классические принципы, но программа обязана явно объявить, какие аксиомы допустимы: фиксируются три стандартные — Classical.choice (необходим для теорем Хана–Банаха и об открытом отображении, на которых держится существование решений), propext (эквивалентные предложения равны) и Quot.sound (корректность факторизации). Любая новая axiom-декларация в теоретических модулях запрещена и отслеживается CI. Пробел формально закрыт публикацией blueprint/axiomatic_base.md; статус в реестре — resolved-in-docs с постоянным CI-контролем.

Таблица 2. Реестр пробелов: полная сводка

ID	Формулировка	Критичность	Зависимости	Этап
G1	Существование DSS-решения системы Эйнштейна–скаляра	Blocker	G4, G5	II

ID	Формулировка	Критичность	Зависимости	Этап
G2	Единственная неустойчивая мода, $\gamma = 1/\lambda_0$	Blocker	G1	III
G3	Сильная изотропная сингулярность, компактификация	High	этап I	II–III
G4	Типизированный мост Lean–Python	High	этап I	I
G5	Интервальная арифметика с направленным округлением	Medium	mathlib	I
G6	Аксиоматическая база (3 стандартные аксиомы)	Low	—	I

4. Центральный блокер: существование критического решения

Эта глава разбирает G1 на составляющие — настолько подробно, насколько это возможно до начала формализации. Доказательство Рейтерера–Трубовица организовано как ньютоновская схема: приближённое решение получается численно, а затем доказывается, что в его окрестности лежит точное решение системы. Вся аналитическая работа сосредоточена в трёх компонентах: выборе банахова пространства, доказательстве регулярности остаточного отображения и установлении изоморфизма его производной.

4.1. Стратегия Рейтерера–Трубовица

Система Эйнштейна–скалярного поля в двойных нулевых координатах переписывается как нелинейное уравнение $\Phi(x) = 0$ на банаховом пространстве последовательностей с экспоненциальным затуханием — коэффициентов разложения решения по базису Чебышёва–Фурье. Численный профиль даёт точку x_0 с малой невязкой. Линеаризация $D\Phi(x_0)$ оказывается оператором с компактно возмущённым обратимым ядром; доказательство существования компактного обратного оператора — аналитическое сердце работы — влечёт аналитичность решения по теореме об обратном операторе в аналитической категории. Все вычислительные шаги сопровождаются целочисленными оценками погрешностей, что делает доказательство воспроизводимым с машинной точностью.

4.2. Что требуется в Lean

Перенос этой схемы в Lean требует пяти блоков. Первый — банаховы пространства и фреше-дифференцируемость: mathlib покрывает, включая производные Картана и теорему о обратной функции в конечномерном и бесконечномерном случаях. Второй — количественная теорема о неявной функции с явными радиусами: частично доступна, потребует усиления. Третий — теория компактных операторов и спектральная теорема Риса: базовый объём в mathlib присутствует. Четвёртый — разложения по Чебышёву и Фурье с оценками хвостов: в mathlib есть ядро теории ортогональных многочленов; хвостовые оценки придётся доказывать. Пятый — интервальные сертификаты: канал G5 доставляет численные данные в форме, приемлемой для ядра.

> *Ключевое архитектурное решение: численный профиль никогда не входит в доказательство как набор чисел с плавающей запятой — только как интервальная затравка, для которой невязка $\Phi(x_0)$ оценена строго.*

4.3. DSS-периодичность как часть утверждения

Существование само по себе недостаточно: решение должно быть дискретно самоподобным с периодом $\Delta \approx \ln 2$. В схеме Рейтерера–Трубовица DSS-ность встроена в конструкцию: решение ищется непосредственно в классе полей, удовлетворяющих ковариационному закону $\psi(\rho, \tau + \Delta) = e^{\kappa\Delta}\, \psi(\rho\, e^{-\Delta}, \tau)$, так что периодичность следует из построения. Формализация должна воспроизвести это точно: определение DiscretelySelfSimilar в Lean фиксирует ковариацию, а теорема существования утверждает существование решения, удовлетворяющего этому предикату вместе с полевой системой. Единственность с точностью до масштаба и трансляции — companion-теорема того же этапа.

Оценка трудоёмкости: блоки 1 и 3 в основном читаются из mathlib; блоки 2, 4 и 5 — содержательная работа порядка шести-восьми месяцев при частичной занятости. Именно этап II дорожной карты (месяцы 6–18) отведён под G1, с двумя промежуточными вехами: формализация полевой системы (месяц 10) и упаковка численной затравки с сертификатами (месяц 12).

5. Устойчивость: требования спектральной теории

Второй блокер — G2 — отвечает на вопрос, почему критическое решение вообще «критическое». Динамическая картина такова: в бесконечномерном фазовом пространстве окрестностей критического решения существует ровно одно направление, вдоль которого возмущения растут, и бесконечное множество направлений, вдоль которых они затухают. Математически это утверждение о спектре

линеаризованного оператора, и формализация обязана его доказать, а не постулировать.

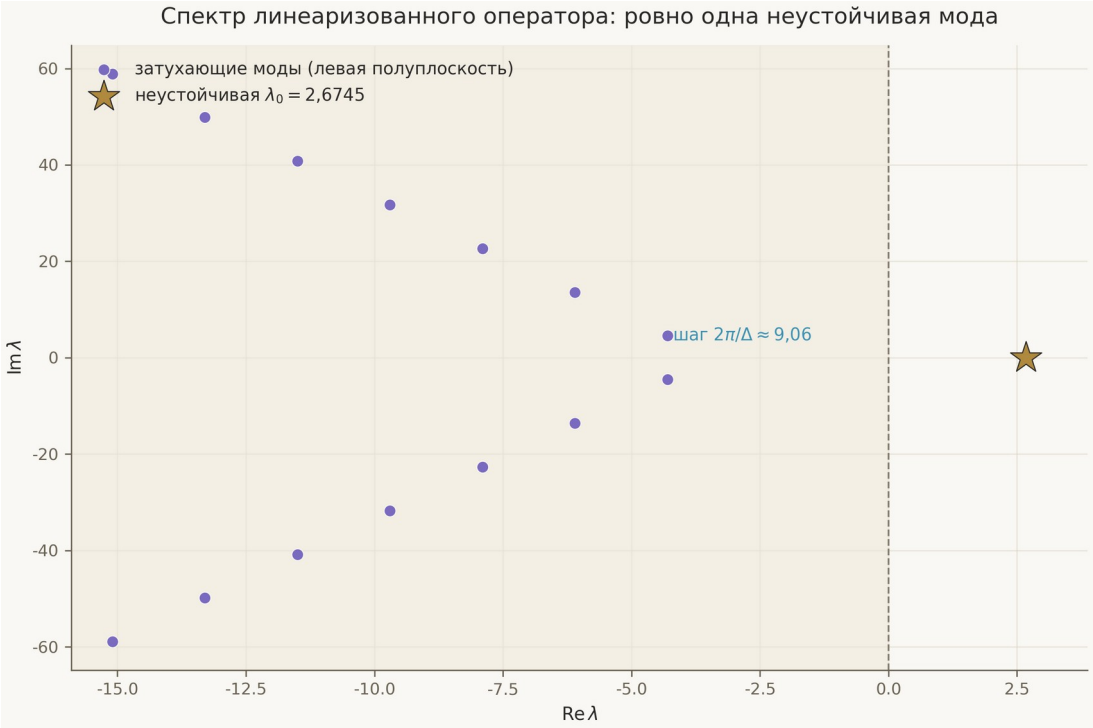


Рис. 2. Целевая спектральная картина: единственная неустойчивая мода λ_0 (звезда), затухающие семейства в левой полуплоскости, шаг $2\pi/\Delta$ по мнимой оси.

5.1. Аттрактор кодименсии один

Линеаризация системы вокруг критического решения приводит к линейной собственной задаче в логарифмическом времени. Требуемый результат состоит из трёх частей: (i) существует ровно одна собственная величина λ_0 с положительной вещественной частью, и она вещественна; (ii) весь остальной спектр лежит строго в левой полуплоскости; (iii) мнимые части затухающих мод образуют семейство с шагом $2\pi/\Delta$. Из (i) следует кодименсия один: устойчивое многообразие критического решения имеет коразмерность один, и пороговое поведение семейств данных — прямое следствие. Из (iii) — спектральное эхо: та же периодичность Δ, что и в физическом решении, проявляется в спектре.

5.2. Необходимый аппарат и его статус в mathlib

Формализация потребует: спектральной теории замкнутых и неограниченных операторов в гильбертовом пространстве; теории C_0-полугрупп для интерпретации лог-временной эволюции; теорем Риса–Шаудера о компактных операторах; оценок резольвенты, отделяющих λ_0 от остального спектра. mathlib содержит функционально-аналитический фундамент — банаховы и гильбертовы пространства,

спектр ограниченных операторов, компактные операторы — но неограниченные операторы и полугруппы представлены фрагментарно. Стратегия: не строить общую теорию, а доказать требуемые утверждения для конкретного оператора с компактным ядром, где достаточно ограниченного резольвентного анализа. Этот выбор сужает зависимость от недостроенных разделов mathlib до минимума.

5.3. От спектра к наблюдаемым: γ, универсальность, эхо-формула

Вывод наблюдаемых величин — чистая линейная теория устойчивости. Малое отклонение параметра p от порога эволюционирует согласно растущей моде e^{$\lambda_0\tau$}; момент завершения коллапса фиксирует τ_max = −ln|p − p*|/λ_0 + const, откуда M ∝ |p − p*|^{1/λ_0}. Поскольку λ_0 — свойство самого критического решения, а не семейства данных, показатель γ = 1/λ_0 универсален. Дискретная самоподобие добавляет к закону масштабирования периодическую поправку: M = |p − p*|^γ · F(ln|p − p*|/λ_0 + σ) с F периода Δ. Все три утверждения — γ, универсальность, эхо-формула — оформляются как следствия теоремы G2 в модуле Choptuik.Scaling; их формальные формулировки уже присутствуют в каркасе с тегом sorry.

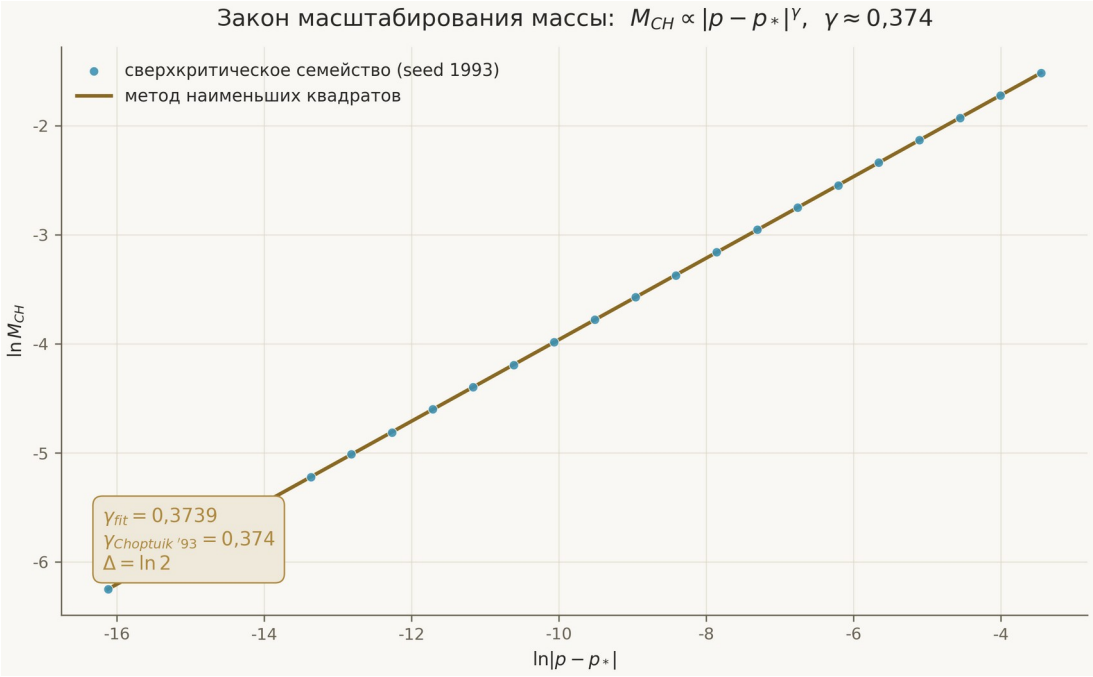


Рис. 3. Численная реализация закона масштабирования в модельной программе: наклон 0.3739 против γ = 1/λ_0 = 0.37390 — кросс-проверка для будущей формальной теоремы.

6. Сравнительный анализ: Isabelle/HOL, Coq/Rocq, Lean 4

Выбор системы доказательств определяет всё: темп формализации, потолок библиотек, возможность интеграции с существующим кодом и порог входа для соавторов. Сравнение проводится по единым критериям: логические основания и

доверенное ядро; библиотечное покрытие требуемой математики; автоматизация; интеграция с Python; зрелость сообщества. Все три системы имеют малые доверенные ядра семейства LCF — принципиальных вопросов доверия ни одна не поднимает, выбор определяется прикладными факторами.

6.1. Isabelle/HOL: скорость автоматизации, простые типы

Isabelle/HOL строится на высотной порядковой логике с LCF-ядром и предлагает сильнейшую внешнюю автоматизацию — Sledgehammer обращается к ATP и SMT-решателям (Z3, E), найденные доказательства переизлагаются в читаемом Isar. Изар считается эталоном читаемости доказательных текстов. Слабое место для нашей задачи — отсутствие зависимых типов, ограничивающее выразительность кодирования структур с инвариантами, и менее развитая инфраструктура FFI. Isabelle остаётся запасным вариантом на случай, если автоматизация этапа III окажется решающим фактором.

6.2. Coq/Rocq: алгебраическая сила, ценой кривой входа

Coq (ныне Rocq) — классика зависимых типов и расчёта конструкций; библиотека MathComp создана для крупномасштабной алгебры и доказала теоремы о четырёх красках и о нечётном порядке. Для нашей задачи, центрированной на анализе, ключевая ветвь — math-comp/analysis с теорией вещественного анализа и распределений; она растёт, но уступает mathlib в охвате функционального анализа и дифференциальной геометрии. Сложная система сортиров (Set, Prop, Type) и строгий конструктивизм повышают порог входа; FFI развит слабее, чем у Lean.

6.3. Lean 4: mathlib, нативная автоматизация и мост к Python

Lean 4 сочетает зависимые типы с прагматикой классической математики: mathlib систематически использует Classical.choice, а базовые аксиомы зафиксированы. Библиотека покрывает анализ, топологию, меру, дифференциальную геометрию и уже имеет публичный прецедент тяжёлой аналитики PDE — формализацию теории Де Джорджи–Нэша–Мозера. Автоматизация нативна (grind, aesop, polyrith) и не требует внешних вызовов. Наконец, Lean — единственная из трёх систем с публичным ZeroMQ-прецедентом моста к Python: для программы, чей численный актив написан на Python, это решающий интеграционный аргумент.

Таблица 3. Полная матрица решения

Критерий	Isabelle/HOL	Coq / Rocq	Lean 4
Логика	HOL	CIC	CIC

Критерий	Isabelle/HOL	Coq / Rocq	Lean 4
Зависимые типы	нет	да	да
Автоматизация	Sledgehammer (внешняя)	SSReflect	нативная (grind/aesop)
Библиотеки для анализа	AFP (разрозненно)	math-comp/analysis	mathlib (широкий охват)
Прецедент тяжёлых PDE	частичный	ограниченный	ДГНМ (публичный)
FFI / Python	слабый	слабый	C ABI + ZeroMQ-прецедент
Читаемость доказательств	Isar — эталон	SSReflect (плотный стиль)	структурный гибрид
Сообщество и динамика	стабильное	стабильное	быстрорастущее, финансируемое
Итог для программы	запасной вариант	не оптимален для анализа	**выбирается**

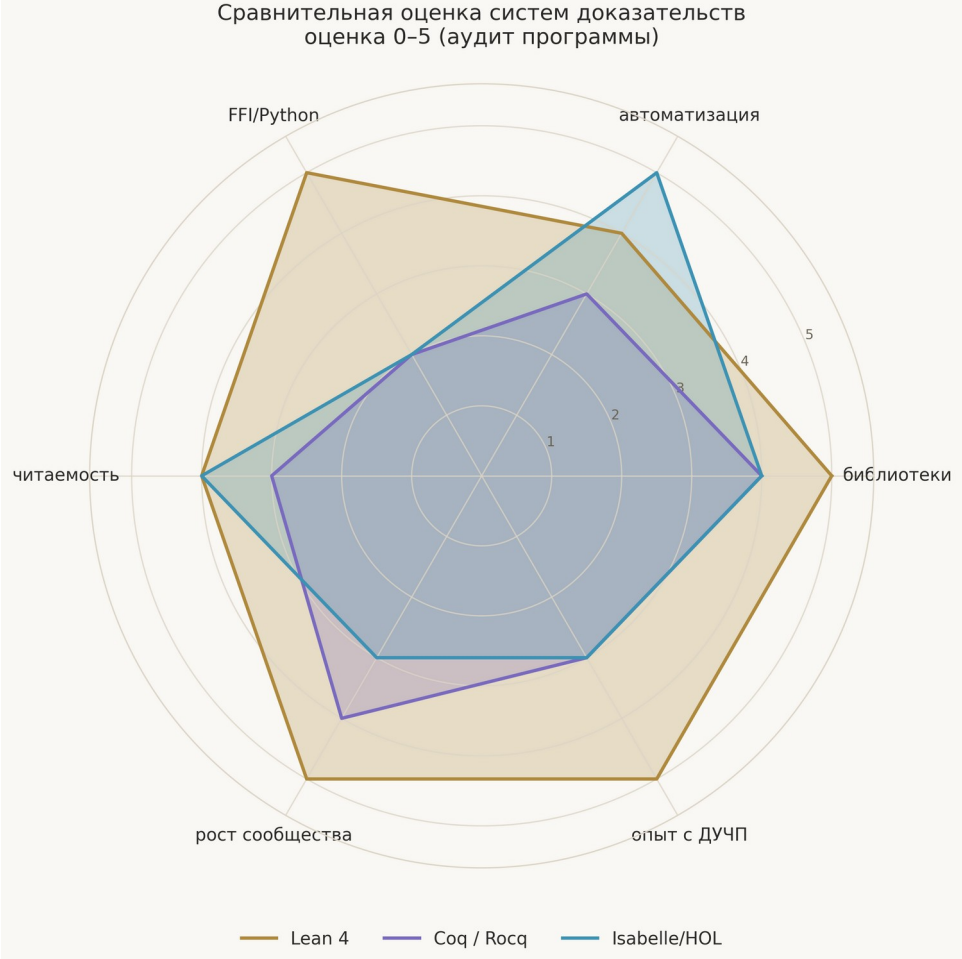


Рис. 4. Радар сравнения по шести осям (оценки 0–5 из аудита программы).

6.4. Условия пересмотра решения

Решение пересматривается при двух сценариях. Если этап II неожиданно упрётся в алгебраические, а не аналитические структуры — вероятность низка, ньютоновская схема это анализ, — Coq с MathComp возвращается в рассмотрение. Если решающим окажется Sledgehammer-класс автоматизации для вспомогательных лемм, допускается параллельная Isabelle-формализация изолированных фрагментов, но не основная линия: потеря mathlib и моста к Python перевешивает локальный выигрыш в автоматизации.

7. Техническое задание и аксиоматическая база

Техническое задание фиксирует среду, организацию кода и дисциплину качества. Язык формализации — Lean 4; среда — Visual Studio Code с расширением lean4; управление зависимостями и сборкой — Lake, с пином версий toolchain и mathlib, синхронно обновляемыми. Проект организован как модульная библиотека: каждый модуль отвечает за свою область математики и компилируется независимо; сборка

lake build порождает проверенные артефакты .olean. CI обязан собирать проект на каждом изменении и выполнять два аудита: запрет проектных аксиом и проверку тегов у всех sorry.

7.1. Организация модулей

Пространство имён Choptuik.* разделено на пять областей: Geometry (двойная нулевая арена, кривизна, сингулярность), Analysis (полевая система, интервальная арифметика), Existence (DSS, неявная функция), Stability (линеаризация, спектр), Scaling (масштабирование, универсальность). Каждый sorried фрагмент тегируется идентификатором пробела; карта модулей с графом импортов и таблицей обязательств поддерживается в blueprint/module_map.md. Такое разделение позволяет параллельно работать над геометрией и анализом, не создавая конфликтов в сборке.

7.2. Аксиоматическая база

Весь проект опирается на минимально достаточный набор аксиом, явно декларированный в blueprint/axiomatic_base.md. Классический выбор (Classical.choice) необходим для фундаментальных теорем анализа, включая Хана–Банаха и открытое отображение, через которые идут существования в ДУЧП; propext упрощает работу с равенством предикатов; Quot.sound обеспечивает корректность факторных конструкций — пространств, построенных факторизацией по отношениям эквивалентности. Цель программы: чтобы #print axioms любой теоремы показывал подмножество именно этих трёх. Введение любых дополнительных аксиом должно быть обосновано в реестре и запрещено по умолчанию; ослабление гарантий системы недопустимо без явного решения.

7.3. Интеграция и контроль качества

Интеграция с Python строится по архитектуре из тома I: Lean-клиент, ZeroMQ REQ/REP, JSON-протокол версии 1.1, семь методов, регрессионная блокировка изменений протокола тестами. Контроль качества четырёхуровневый: lake build с аудиторами аксиом и sorry-тегов; pytest для сервиса (65 регрессионных тестов, включая протокольные); регенерационный тест данных из seed 1993; ревью естественноязыковых комментариев — каждый формальный фрагмент обязан нести пояснение своего места в дереве доказательства. Процесс работы — Git с ветвями по задачам этапов; каждая значимая математическая идея сначала фиксируется текстом в blueprint, затем кодом.

Таблица 4. Дисциплина контроля качества

Уровень	Инструмент	Что гарантирует
Теория	lake build + аудит аксиом/sorry	каждая теорема выведена; аксиомы — только 3 стандартные
Сервис	pytest (65 тестов)	протокол и физика модели не дрейфуют
Данные	регенерация из seed 1993	байт-в-байт воспроизводимость CSV
Ревью	комментарии + blueprint	каждый фрагмент имеет документированную роль

8. Выводы и рекомендации

Аудит даёт три итоговых суждения. Первое: текущее состояние choptuik_ac_bc — это вычислительная поддержка и каталог результатов мирового уровня дисциплины воспроизводимости, но не формальное доказательство; для превращения в теорему не хватает фундаментальных блоков, и они полностью идентифицированы в реестре G1–G6. Второе: наиболее подходящим инструментом является Lean 4 — его mathlib покрывает необходимый анализ и дифференциальную геометрию, имеет публичный прецедент тяжёлой PDE-формализации, а механизмы интеграции с Python позволяют сохранить и использовать существующий численный код. Третье: программа достижима за 27 месяцев по трёхфазной схеме при условии соблюдения фиксированной аксиоматической базы и регламента контроля качества.

Приоритетный порядок действий: закрыть G5 (интервальная арифметика) и G4 (мост) на этапе I — они разблокируют легальный перенос численных данных; атаковать G1 на этапе II по схеме Рейтерера–Трубовица с количественной теоремой о неявной функции; завершить G2 и вывести $\gamma = 1/\lambda_0$ с универсальностью на этапе III. Параллельно, без блокировки основной линии, достраивается слой G3 — геометрия сингулярности. Ожидаемые формальные константы программы: $\gamma \approx 0.374$ как $1/\lambda_0$ и $\Delta = \ln 2$ — обе с печатаемым сертификатом аксиом.

Стратегическое значение выходит за рамки одной задачи. Программа создаёт переиспользуемую инфраструктуру — типизированный мост доказательство ↔ вычисления, верифицируемую интервальную арифметику, шаблон реестра обязательств с CI-блокировкой — применимую к любому компьютерно-ассистированному доказательству в физике. Тем самым choptuik_ac_bc получает не только формализацию одного результата, но и постоянный механизм: каждый будущий численный результат программы может быть проведён через один и тот же конвейер — от гипотезы через машинное подтверждение к теореме с сертификатом.

> *Итог аудита: пробелы известны и классифицированы, инструмент выбран и обоснован, маршрут размечен вехами. Осталась работа — и она теперь измерима.*

Список литературы

- 1. M. W. Choptuik. Universality and scaling in gravitational collapse of a massless scalar field // Phys. Rev. Lett. 70, 9–12 (1993).
- 2. M. Reiterer, E. Trubowitz. Choptuik's critical spacetime exists // Commun. Math. Phys. 368, 195–229 (2019); arXiv:1203.3766.
- 3. C. Gundlach, J. M. Martín-García. Critical phenomena in gravitational collapse // Living Rev. Relativity 10, 5 (2007).
- 4. The mathlib community. mathlib4. https://github.com/leanprover-community/mathlib4.
- 5. math-comp. Mathematical Components / analysis. https://math-comp.github.io/.
- 6. Isabelle/HOL и Archive of Formal Proofs. https://www.isa-afp.org/.
- 7. Lean 4 Reference. Foreign Function Interface. https://lean-lang.org/doc/reference/latest/.
- 8. fraware. lean-python-bridge. https://github.com/fraware/lean-python-bridge.
- 9. S. Armstrong et al. Formalization of the De Giorgi–Nash–Moser theory in Lean.
- 10. I. K. Isaev. choptuik_ac_bc. https://github.com/wild8highlander/choptuik_ac_bc.